Data Science Salaries analysis

Problem Description :

Problem Description:

As organizations increasingly rely on data-driven decision-making, data science has emerged as a critical domain. Data scientists, data engineers, machine learning engineers, and other related roles play a crucial role in extracting insights from data and developing intelligent systems. Salaries in the data science field can vary significantly based on factors such as job category, experience level, employment type, company location, and more. Analyzing data science salaries is essential to understand industry trends, identify factors that influence compensation, and make informed career decisions.

Background Information:

The Data Science Salaries Analysis project aims to provide valuable insights into the salaries of professionals in the data science domain. The project utilizes a dataset containing information on data science-related roles, including job titles, experience levels, employment types, salaries, company sizes, and remote work ratios. The dataset may have been obtained from job portals, company records, or surveys.

Dataset Information:

The dataset consists of the following columns:

1. **job_title:** The specific job title or role of the individual (e.g., Data Scientist, Data Engineer, Machine Learning Engineer).
2. **experience_level:** The experience level of the employee (e.g., Senior, Entry level, Executive level).

3. **employment_type:** The type of employment (e.g., Full-time, Part-time, Freelancer).
4. **remote_ratio:** The percentage of remote work offered by the employer (e.g., On-Site, Half-Remote, Full-Remote).
5. **company_size:** The size of the company (e.g., SMALL, MEDIUM, LARGE).
6. **company_location:** The location of the company.
7. **employee_residence:** The location of the employee's residence.
8. **salary_currency:** The currency in which the salary is provided (e.g., USD, EUR, GBP).
9. **salary_in_usd:** The salary amount in USD.
10. **work_year:** The year in which the salary information was recorded.

Project Goals:

- The main objectives of the Data Science Salaries Analysis project are as follows:
- **Data Cleaning and Preparation**: Handle missing data, remove duplicates, and transform categorical variables for analysis.
- **Inflation-Adjusted Salary Calculation:** Adjust the salaries for inflation to make meaningful comparisons across different years.
- **Job Category Analysis:** Analyze the distribution of job categories and compare salaries across different employment types.
- **Salary Distribution Analysis:** Examine the distribution of adjusted salaries and identify any outliers.
- **Salary Trends by Year and Location:** Investigate how salaries vary over the years and across different company locations and employee residences.
- **Data Preprocessing for Machine Learning**: Prepare the data for machine learning models by encoding categorical variables and scaling numerical features.
- **t-SNE Visualization:** Visualize the data in a 2D space using t-SNE to gain insights into salary clusters.
- **Outlier Detection and Removal:** Use the Interquartile Range (IQR) method to remove outliers from the dataset.
- **Machine Learning Model Selection and Evaluation:** Train and evaluate various machine learning models to predict salary categories.
- **Best Categories for Maximized Salary:** Identify the best categories (e.g., job category, experience level) that result in higher adjusted salaries.

Overall, the Data Science Salaries Analysis project aims to provide a comprehensive understanding of data science salaries, helping professionals in the field make informed career decisions and assisting organizations in salary benchmarking and talent acquisition. The project involves data exploration, visualization, preprocessing, and machine learning techniques to extract valuable insights from the dataset.

Possible Framework :

1. **Import Libraries and Data:**
- Import the necessary Python libraries, such as pandas, numpy, matplotlib, seaborn, plotly, scipy, and scikit-learn.
- Load the dataset 'data.csv' into a pandas DataFrame.
2. **Data Exploration and Cleaning:**
- Perform initial data exploration to understand the structure of the dataset.
- Check for missing values and calculate the percentage of missing data for each column.
- Remove duplicate rows from the dataset if any.
- Update job titles, employment types, company sizes, and remote work ratios with more meaningful labels.
- Define a function to assign broader job categories based on job titles.
3. **Inflation-Adjusted Salary Calculation:**
- Define dictionaries containing annual inflation rates for the US and global economies from 2019 to 2022.
- Implement a function to adjust salaries for inflation and create a new column 'adjusted_salary'.
4. **Job Category Analysis:**
- Calculate the percentage of job titles belonging to each broader job category.
- Create a horizontal bar plot to visualize the distribution of job categories.
5. **Salary Distribution Analysis:**
1. Plot a box plot to compare the salary distribution across different employment types.
2. Calculate the mean, standard deviation, skewness, and kurtosis of the adjusted salary distribution.
3. Plot a histogram with a normal fit and a probability plot (Q-Q plot) to assess data normality.
6. **Salary Trends by Year and Location:**
- Create a heatmap to display the median salary by job category and year.
- Create a scatter plot to compare employee residence and company location against adjusted salary.
- Use the pycountry library to convert country codes to country names.
7. **Data Preprocessing for Machine Learning:**

- Encode categorical variables using LabelEncoder.
- Scale numerical features (adjusted_salary) using MinMaxScaler.

8. **t-SNE Visualization:**

- Apply t-Distributed Stochastic Neighbor Embedding (t-SNE) to reduce data dimensions to 2D.
- Create a scatter plot to visualize salary clusters in the reduced feature space.

9. **Outlier Detection and Removal:**

- Use the Interquartile Range (IQR) method to detect and remove outliers from the dataset.

10. **Machine Learning Model Selection and Evaluation:**

- Split the preprocessed data into training and testing sets.
- Define a list of machine learning models, including Logistic Regression, Random Forest, and Gradient Boosting.
- Train each model and evaluate its accuracy on the test set.
- Select the model with the highest accuracy as the best model.

11. **Confusion Matrix Visualization:**

- Create a heatmap of the confusion matrix to visualize the performance of the best model.

12. **Classification Report:**

- Generate a classification report to assess the precision, recall, and F1-score for each salary category.

13. **Best Categories for Maximized Salary:**

- Identify the best categories (e.g., job category, experience level) that lead to higher adjusted salaries.

14. **Data Visualization:**

- Visualize average salaries by company location and employee residence using choropleth maps.

15. **Conclusion:**

- Summarize the key findings and insights from the data analysis and machine learning models.

16. **Future Work and Recommendations:**

- Suggest potential areas for further analysis and improvements in the data collection process.

17. **Code Refactoring and Optimization:**

- Review the code for readability, efficiency, and maintainability.

- Add comments and docstrings to enhance code understanding.

18. Final Remarks:

- Provide a closing statement and appreciation for exploring data science salaries.

Code Explanation :

*If this section is empty, the explanation is provided in the .ipynb file itself.

1. Importing Libraries and Data: The code starts by importing essential Python libraries like pandas, numpy, matplotlib, seaborn, and more. These libraries are essential tools that data scientists use to work with data and visualize it. The data.csv file is loaded into a pandas DataFrame, which is like a table with rows and columns to store and manipulate data.

2. Data Exploration and Cleaning: Data scientists always explore the data first to understand its structure and spot any issues. The code checks for missing values and calculates the percentage of missing data for each column. It also removes any duplicate rows from the dataset to ensure data accuracy. Then, it updates certain columns like job titles and employment types with more meaningful labels to make them easier to understand.

3. Inflation-Adjusted Salary Calculation: This part of the code handles inflation in salary data. Inflation is when the general prices of goods and services increase over time, leading to a decrease in the purchasing power of money. The code defines dictionaries containing annual inflation rates for the US and global economies. It then implements a function to adjust salaries for inflation based on the year of the data, using the inflation rates from the dictionaries. The adjusted salaries are stored in a new column called 'adjusted_salary'.

4. Job Category Analysis: To analyze the job categories, the code calculates the percentage of job titles belonging to each broader category. It then creates a horizontal bar plot to visually represent the distribution of job categories and how many job titles fall into each category.

5. Salary Distribution Analysis: This section focuses on the distribution of salaries. The code plots a box plot to compare the salary distribution across different employment types. It calculates statistical measures like mean, standard deviation, skewness, and kurtosis to understand the shape of the salary distribution. Additionally, it plots a histogram and a normal fit to visualize the frequency of salary values and whether they follow a normal distribution.

6. Salary Trends by Year and Location: Here, the code explores the salary trends based on the year and the location of the companies and employees. It creates a heatmap to show the median salary by job category and year, allowing us to observe salary trends over time. The code also plots a scatter plot to compare employee residence and company location against adjusted salary, giving insights into how location affects salaries.

7. Data Preprocessing for Machine Learning: Machine learning models require numerical data for training. So, this part of the code preprocesses the data by encoding categorical variables (like job category and experience level) into numerical values using LabelEncoder. It also scales the numerical features, like adjusted salary, using MinMaxScaler to ensure all features have similar scales.

8. t-SNE Visualization: t-SNE is a technique used to visualize high-dimensional data in 2D or 3D space. In this code, t-SNE is applied to reduce the dimensions of the data to 2D. This way, we can visualize how the salaries cluster in a lower-dimensional space.

9. Outlier Detection and Removal: Outliers are extreme values that can significantly affect the statistical analysis. In this section, the code uses the Interquartile Range (IQR) method to detect and remove outliers from the dataset. This ensures that the analysis is more robust and not heavily influenced by extreme values.

10. Machine Learning Model Selection and Evaluation: Machine learning models are used to predict salary categories based on other features. The code defines a list of models like Logistic Regression, Random Forest, and Gradient Boosting. It then trains each model using the preprocessed data and evaluates their performance using accuracy on the test set. The model with the highest accuracy is selected as the best model for salary prediction.

11. Confusion Matrix Visualization: The code generates a heatmap of the confusion matrix to visualize how well the best model performed in predicting each salary category. The confusion matrix shows how many predictions were correct and how many were incorrect for each category.

12. Classification Report: The classification report provides a more detailed assessment of the model's performance. It includes metrics like precision, recall, and F1-score for each salary category. These metrics help us understand how well the model performs for different salary levels.

13. Best Categories for Maximized Salary: This part of the code identifies the best categories (e.g., job category, experience level) that lead to higher adjusted salaries. It finds the categories where the mean salary is highest.

14. Data Visualization: To make the analysis more insightful, the code creates choropleth maps to visualize average salaries by company location and employee residence. These maps help us understand how salaries vary across different countries.

15. Conclusion: In the conclusion section, the code could summarize the key findings and insights gained from the data analysis. This might include salary trends, the impact of location on salaries, and the best job categories for higher salaries.

16. Future Work and Recommendations: To improve the analysis further, the code might suggest potential areas for future research and improvements in data collection. It could also provide recommendations to companies or job seekers based on the analysis.

17. Code Refactoring and Optimization: Finally, the code might be reviewed for readability, efficiency, and maintainability. Comments and docstrings could be added to enhance code understanding for others who might read or use the code in the future.

Future Work :

1. Importing Libraries and Data: The code starts by importing essential Python libraries like pandas, numpy, matplotlib, seaborn, and more. These libraries are essential tools that data scientists use to work with data and visualize it. The data.csv file is loaded into a pandas DataFrame, which is like a table with rows and columns to store and manipulate data.

2. Data Exploration and Cleaning: Data scientists always explore the data first to understand its structure and spot any issues. The code checks for missing values and calculates the percentage of missing data for each column. It also removes any duplicate rows from the dataset to ensure data accuracy. Then, it updates certain columns like job titles and employment types with more meaningful labels to make them easier to understand.

3. Inflation-Adjusted Salary Calculation: This part of the code handles inflation in salary data. Inflation is when the general prices of goods and services increase over time, leading to a decrease in the purchasing power of money. The code defines dictionaries containing annual inflation rates for the US and global economies. It then implements a function to adjust salaries for inflation based on the year of the data, using the inflation rates from the dictionaries. The adjusted salaries are stored in a new column called 'adjusted_salary'.

4. Job Category Analysis: To analyze the job categories, the code calculates the percentage of job titles belonging to each broader category. It then creates a horizontal bar plot to visually represent the distribution of job categories and how many job titles fall into each category.

5. Salary Distribution Analysis: This section focuses on the distribution of salaries. The code plots a box plot to compare the salary distribution across different employment types. It calculates statistical measures like mean, standard deviation, skewness, and kurtosis to understand the shape of the salary distribution. Additionally, it plots a histogram and a normal fit to visualize the frequency of salary values and whether they follow a normal distribution.

6. Salary Trends by Year and Location: Here, the code explores the salary trends based on the year and the location of the companies and employees. It creates a

heatmap to show the median salary by job category and year, allowing us to observe salary trends over time. The code also plots a scatter plot to compare employee residence and company location against adjusted salary, giving insights into how location affects salaries.

7. Data Preprocessing for Machine Learning: Machine learning models require numerical data for training. So, this part of the code preprocesses the data by encoding categorical variables (like job category and experience level) into numerical values using LabelEncoder. It also scales the numerical features, like adjusted salary, using MinMaxScaler to ensure all features have similar scales.

8. t-SNE Visualization: t-SNE is a technique used to visualize high-dimensional data in 2D or 3D space. In this code, t-SNE is applied to reduce the dimensions of the data to 2D. This way, we can visualize how the salaries cluster in a lower-dimensional space.

9. Outlier Detection and Removal: Outliers are extreme values that can significantly affect the statistical analysis. In this section, the code uses the Interquartile Range (IQR) method to detect and remove outliers from the dataset. This ensures that the analysis is more robust and not heavily influenced by extreme values.

10. Machine Learning Model Selection and Evaluation: Machine learning models are used to predict salary categories based on other features. The code defines a list of models like Logistic Regression, Random Forest, and Gradient Boosting. It then trains each model using the preprocessed data and evaluates their performance using accuracy on the test set. The model with the highest accuracy is selected as the best model for salary prediction.

11. Confusion Matrix Visualization: The code generates a heatmap of the confusion matrix to visualize how well the best model performed in predicting each salary category. The confusion matrix shows how many predictions were correct and how many were incorrect for each category.

12. Classification Report: The classification report provides a more detailed assessment of the model's performance. It includes metrics like precision, recall, and F1-score for each salary category. These metrics help us understand how well the model performs for different salary levels.

13. Best Categories for Maximized Salary: This part of the code identifies the best categories (e.g., job category, experience level) that lead to higher adjusted salaries. It finds the categories where the mean salary is highest.

14. Data Visualization: To make the analysis more insightful, the code creates choropleth maps to visualize average salaries by company location and employee residence. These maps help us understand how salaries vary across different countries.

15. Conclusion: In the conclusion section, the code could summarize the key findings and insights gained from the data analysis. This might include salary trends, the impact of location on salaries, and the best job categories for higher salaries.

16. Future Work and Recommendations: To improve the analysis further, the code might suggest potential areas for future research and improvements in data collection. It could also provide recommendations to companies or job seekers based on the analysis.

17. Code Refactoring and Optimization: Finally, the code might be reviewed for readability, efficiency, and maintainability. Comments and docstrings could be added to enhance code understanding for others who might read or use the code in the future.

Concept Explanation :

The Fabulous World of Machine Learning Algorithms: A Tale of Predicting Data Science Salaries

Once upon a time in the magical land of Data Science, there was a quest to unlock the secrets of predicting fabulous salaries for data scientists. Our brave adventurers sought the help of mystical beings known as "Machine Learning Algorithms" to aid them on their journey.

Logistic Regression - The Friendly Predictor ⍰

Our first encounter was with Logistic Regression, the cute little mouse of algorithms. Don't be fooled by its size, though! Logistic Regression is excellent at dealing with binary decisions, like predicting whether a data scientist earns a "high" or "low" salary.

Imagine you are hosting a fabulous dinner party, and you want to predict whether your guests will wear elegant suits or casual attire. Logistic Regression would analyze past party data and learn the trends to help you decide the dress code. It would say, "Oh dear, I see most guests wear elegant suits to your parties, so I predict 'elegant suit' for the new guests!"

Random Forest - The Dazzling Dream Team ⍰⍰

Next on our journey was the enchanting Random Forest, a majestic group of decision trees working together. This forest is full of diverse trees, each one making its predictions. They vote on the final decision, like a democratic woodland council!

Imagine you're exploring a mystical forest with many paths. Each tree in the Random Forest is like a guide, suggesting a different route. The final decision is made by tallying the votes of all the guides, and the path with the most votes becomes the official prediction. Talk about teamwork!

Gradient Boosting - The Wizard of Wisdom ⍰♂✨

As we ventured deeper into the data jungle, we met the all-knowing Gradient Boosting, a wise and powerful wizard. This algorithm is a master of combining weak learners (simple models) to create a mighty prediction force.

Imagine you are on a magical quest to solve a riddle. Each weak learner in Gradient Boosting is like a humble riddle solver. They take turns improving upon the previous answer, making steady progress toward the final solution. In the end, their collective wisdom creates a powerful riddle-solving team!

The Quest for Accuracy ⍰

Throughout our adventure, we sought the perfect algorithm to predict data science salaries accurately. Each algorithm brought its unique powers to the table, and we tested them on our fabulous dataset. The decision was not easy, as each had strengths and weaknesses. But in the end, one algorithm stood out as the true champion!

The Grand Winner - Our Superstar Model! ⍰

After much analysis and training, our superstar model emerged victorious - the magnificent Gradient Boosting! Its impressive ability to combine multiple weak learners made it the ideal choice for predicting data science salaries with outstanding accuracy.

So, with the help of Logistic Regression, Random Forest, and the ultimate wisdom of Gradient Boosting, our adventurers conquered the quest for predicting data science salaries, providing valuable insights to job seekers and employers alike in the fabulous world of Data Science! ⍰⍰

Note: Remember, these algorithms are not just magical tools; they are powerful tools that require responsible use and ethical considerations when dealing with real-world data. Now go forth and embark on your own data science adventures, and may the power of machine learning be with you! ⍰⍰

Exercise Questions :

1. Question: How did you handle missing data in the dataset, and what was the proportion of missing values in each column?

Answer: In the project, missing data was handled by calculating the percentage of missing values in each column. The data_na DataFrame was used to store the missing ratio, and columns with missing data above a certain threshold were displayed.

2. Question: Explain the process of assigning broader categories to job titles (e.g., Data Engineering, Data Science) in the dataset. What were the criteria for such assignments?

Answer: The function assign_broader_category was used to map specific job titles to broader categories like "Data Engineering" or "Data Science." The criteria for these assignments were predefined lists of job titles that matched each category. For example, if a job title matched any of the titles in the "Data Engineering" list, it was assigned to that category.

3. Question: Describe how inflation-adjusted salaries were calculated in the dataset. What years were considered, and which inflation rates were used for the calculation?

Answer: The function adjust_salary was responsible for calculating inflation-adjusted salaries. It took the original salary, the year of the salary, and the salary currency as input. It iterated through the years from the salary year up to 2022 and applied the appropriate inflation rate (either US or global) based on the currency. The adjusted salary was calculated by multiplying the original salary with the inflation rate for each year.

4. Question: What was the purpose of using the t-SNE algorithm in the project, and how did it help visualize the data?

Answer: t-SNE (t-distributed Stochastic Neighbor Embedding) was used to visualize the high-dimensional dataset in a 2-dimensional space. It helped explore the patterns and clusters in the dataset and understand how the data points (salary records) were distributed in relation to each other. By plotting the t-SNE results, the project visualized

the salary data points based on their dimensions and created a scatter plot with colors representing different salary ranges.

5. Question: Explain the feature scaling methods used in the project. Why was feature scaling important, and what impact did it have on the machine learning models' performance?

Answer: StandardScaler and MinMaxScaler were used for feature scaling in the project. Feature scaling was essential because it transformed the numerical features to a common scale, preventing any bias due to varying magnitudes. StandardScaler scaled features to have zero mean and unit variance, while MinMaxScaler scaled features to a given range (usually [0, 1]). Scaling helped improve the performance of machine learning models by making them more robust to feature magnitudes and improving convergence during training.

6. Question: Describe the purpose of using a correlation matrix in the project and how it helped in analyzing the relationships between features.

Answer: The correlation matrix was used to understand the relationships between different features (columns) in the dataset. It showed the pairwise correlations between all numerical features. Positive correlations indicated that two features increased together, while negative correlations indicated an inverse relationship. Analyzing the correlation matrix helped identify highly correlated features, which could impact model performance and multicollinearity.

7. Question: Explain the purpose of using the IQR method to handle outliers in the dataset. How did it impact the dataset and the machine learning models' results?

Answer: The Interquartile Range (IQR) method was used to detect and remove outliers from the dataset. Outliers are extreme data points that deviate significantly from the majority of the data. By removing outliers, the dataset became more robust and less sensitive to extreme values, leading to better model generalization and performance.

8. Question: How were the categorical variables encoded in the dataset, and why was it necessary?

Answer: Categorical variables were encoded using LabelEncoder. This transformation converted categorical variables (e.g., job categories, experience levels) into numerical representations. Machine learning models require numerical inputs, so encoding was

necessary to represent categorical data in a format that the models could understand and process.

9. Question: Explain the purpose of using the confusion matrix and classification report when evaluating the machine learning models' performance.

Answer: The confusion matrix provided a summary of the model's predictions and their accuracy by comparing them with the actual target values. It showed the number of true positives, true negatives, false positives, and false negatives. The classification report further provided metrics like precision, recall, and F1-score for each class. These evaluation metrics helped assess the model's performance on different salary range predictions, allowing us to understand the model's strengths and weaknesses.

10. Question: Describe the final step of selecting the best machine learning model for salary prediction. What was the evaluation criterion, and how did you determine the model with the highest accuracy?

Answer: The final step involved training and evaluating three different machine learning models: Logistic Regression, Random Forest, and Gradient Boosting. The evaluation criterion was accuracy, which measures the percentage of correct predictions made by the model. The models were trained on the training dataset and evaluated on the test dataset. The model with the highest accuracy on the test dataset was selected as the best performer for salary prediction.